

Datenanalyse-Agent mit LangGraph

Stufe 1 – Linearer Graph mit ReAct-Zyklus

Architektur-Review & Funktionsbeschreibung

April 2026

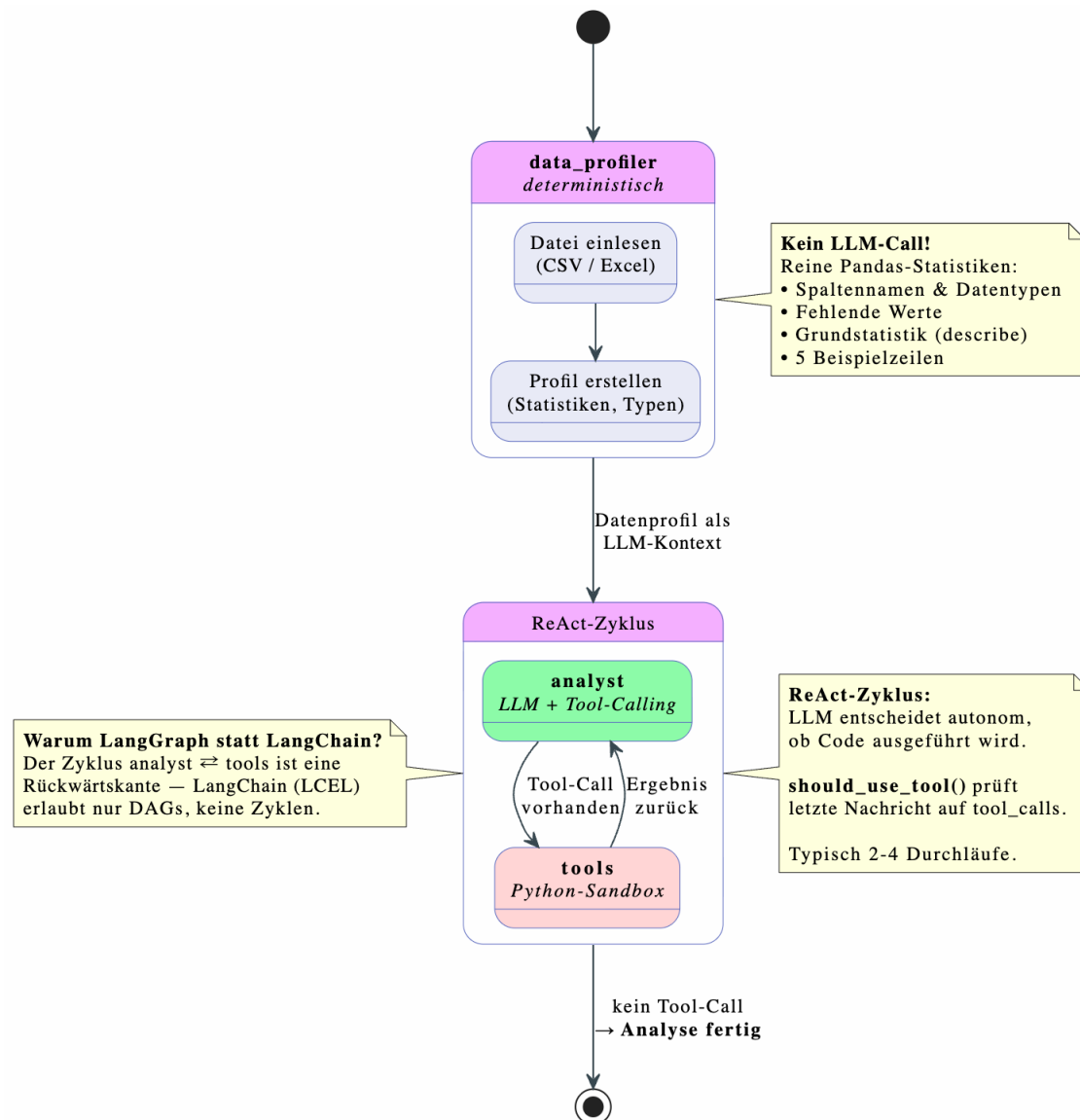
1 Überblick

Dieses Dokument beschreibt den Aufbau und die Funktionsweise eines agentischen Datenanalyse-Assistenten, implementiert als Jupyter Notebook. Der Agent nimmt eine CSV- oder Excel-Datei entgegen und führt autonom eine explorative Datenanalyse durch – von der Strukturerkennung über statistische Auswertungen bis hin zu Visualisierungen und einem zusammenfassenden Bericht.

Technologie-Stack:

- LangGraph (Zustandsgraph),
- LangChain (LLM-Integration),
- Qwen-LLM über DeepInfra-API,
- Pandas, Matplotlib, Seaborn,
- Gradio (Web-UI).

2 Architektur



Der Agent ist als linearer StateGraph mit integriertem ReAct-Zyklus aufgebaut.

Der Kontrollfluss folgt dem Muster:

```
START → data_profiler → analyst ⇌ tools → END
```

Der Zyklus zwischen analyst und tools ist der entscheidende Unterschied zu einer einfachen LangChain-Chain: LangChain (LCEL) erlaubt nur gerichtete azyklische Graphen (DAGs), keine Rückwärtskanten.

LangGraph erlaubt Zyklen, weil es ein zustandsbehafteter Graph ist.

2.1 Knotenstruktur

Knoten	Typ	Aufgabe
data_profiler	Deterministisch	Datei einlesen, Struktur erfassen, Statistiken berechnen – kein LLM-Call
analyst	LLM + Tool-Calling	Analyseplan erstellen, Python-Code generieren und ausführen lassen
tools	ToolNode (Sandbox)	Python-Code sicher ausführen (pandas, matplotlib, seaborn, numpy)

2.2 Kantenlogik

Der Graph verwendet zwei Kantentypen:

- Feste Kanten (add_edge): START → data_profiler → analyst, sowie tools → analyst (Rückführung)
- Konditionale Kante (add_conditional_edges): Vom analyst-Knoten entscheidet die Funktion should_use_tool, ob der LLM einen Tool-Call gemacht hat (weiter zu tools) oder ob er fertig ist (weiter zu END)

3 Komponenten im Detail

3.1 State-Definition

Der State ist ein TypedDict mit drei Feldern, das als zentrales Gedächtnis zwischen allen Knoten dient:

- `file_path` (str): Pfad zur hochgeladenen Datendatei
- `data_profile` (dict): Strukturiertes Datenprofil aus dem Profiler-Knoten
- `messages` (Annotated[list, add_messages]): LLM-Konversation mit Reducer

Der Reducer `add_messages` ist entscheidend: Ohne ihn würde jeder Knoten die gesamte Nachrichtenliste überschreiben. Mit Reducer werden neue Nachrichten angehängt, sodass die Konversationshistorie (System-Prompt, LLM-Antwort, Tool-Ergebnis usw.) kontinuierlich wächst.

3.2 Tool: Python-Sandbox (`execute_python`)

Eine mit `@tool` dekorierte Funktion, die Python-Code in einem eingeschränkten Namespace ausführt. Der Namespace enthält nur `pandas`, `numpy`, `matplotlib` und `seaborn`. Stdout und stderr werden abgefangen und als String zurückgegeben. Fehler werden gefangen und als Fehlertext zurückgegeben, damit der LLM seinen Code selbständig korrigieren kann.

Sicherheitshinweis: In einer Produktionsumgebung würde der Code in einem Docker-Container oder einer E2B-Sandbox ausgeführt werden. Der eingeschränkte Namespace ist nur für Lernzwecke ok.

3.3 Data Profiler

Ein rein deterministischer Knoten ohne LLM-Beteiligung. Er liest die Datei ein (CSV mit Fallback von Komma auf Semikolon als Trennzeichen, oder Excel) und erstellt ein strukturiertes Profil mit folgenden Informationen:

- Dateiname, Zeilen- und Spaltenanzahl
- Spaltennamen mit Datentypen
- Fehlende Werte pro Spalte
- Numerische und kategoriale Spalten
- Grundstatistik (`describe`) und 5 Beispielzeilen

Das Profil wird dem LLM als Kontext mitgegeben. So versteht er die Datenstruktur, ohne Millionen Rohdaten-Tokens verarbeiten zu müssen – eine bewusste Architekturentscheidung zugunsten von Kosten und Geschwindigkeit.

3.4 Analyst (ReAct-Agent)

Der Kernknoten des Agenten. Beim ersten Aufruf wird ein System-Prompt mit dem Datenprofil und Handlungsanweisungen erstellt. Bei Folgeaufrufen (nach einem Tool-Call) werden die bestehenden Messages wiederverwendet – das LLM sieht die gesamte bisherige Konversation & Tool-Ergebnisse. Das LLM wird über `bind_tools` mit dem `execute_python`-Tool verbunden & entscheidet autonom:

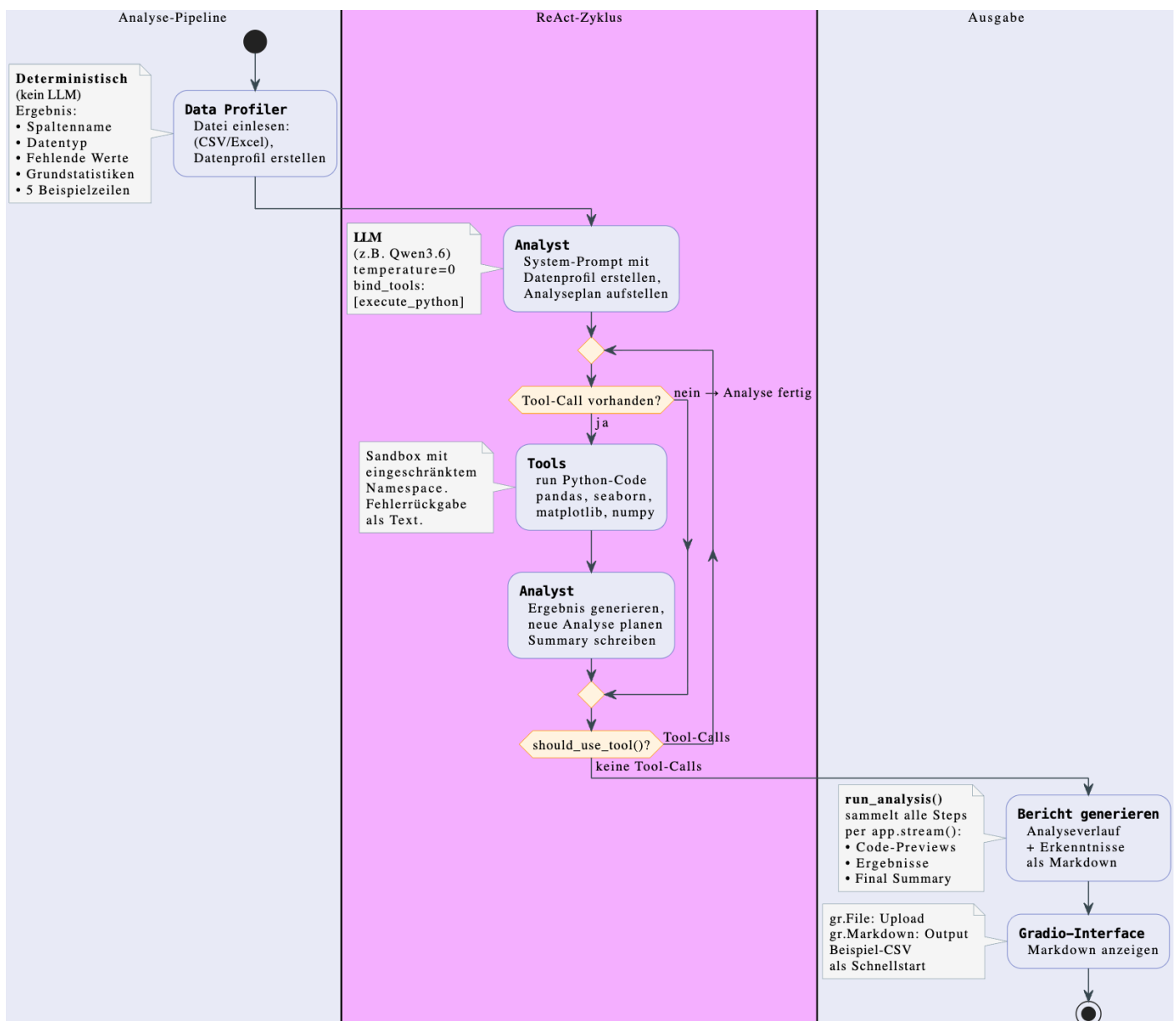
- Tool-Call generieren: Er will Code ausführen → weiter zu tools
- Reine Textantwort: Er ist fertig → weiter zu END

Dies ist der ReAct-Zyklus (Reason + Act): Der LLM steuert den Kontrollfluss des Graphen selbständig.

3.5 Routing-Funktion (`should_use_tool`)

Die konditionale Kantenfunktion prüft die letzte Nachricht des LLM. Enthält sie `tool_calls`, gibt die Funktion den String "tools" zurück, andernfalls END. Damit steuert der LLM indirekt den Graphverlauf.

4 Ablauf einer Analyse



Ein vollständiger Durchlauf des Agenten umfasst folgende Schritte:

1. Der Nutzer lädt eine CSV- oder Excel-Datei hoch (via Gradio-Interface oder direkt als Pfad).
2. Der `data_profiler` liest die Datei ein und erstellt ein strukturiertes Profil (kein LLM-Call).
3. Der `analyst` erhält das Profil, plant die erste Analyse und generiert einen Tool-Call mit Python-Code.
4. `should_use_tool` erkennt den Tool-Call und leitet zum `tools`-Knoten weiter.
5. Der `ToolNode` führt den Code aus und schreibt das Ergebnis als `ToolMessage` in den State.
6. Der `analyst` sieht das Ergebnis, plant weitere Analysen oder generiert neuen Code.
7. Schritte 4–6 wiederholen sich (typisch 2–4 Zyklen).
8. Der `analyst` antwortet mit reinem Text (Zusammenfassung) – keine Tool-Calls mehr.
9. `should_use_tool` erkennt das Fehlen von Tool-Calls und routet zu END.
10. Der Markdown-Bericht wird im Gradio-Interface angezeigt.

5 Streaming & Berichterstellung

Die Funktion `run_analysis` nutzt `app.stream()` mit `stream_mode="updates"`, um bei jedem Graphschritt ein Dictionary zu erhalten. Für jeden Knotentyp werden spezifische Informationen extrahiert:

- `data_profiler`: Zeilen-/Spaltenanzahl, numerische und kategorische Spalten, fehlende Werte
- `analyst`: Code-Previews der Tool-Calls oder die finale Zusammenfassung
- `tools`: Ausgabe-Previews der Code-Ausführung

Diese Informationen werden zu einem zweistufigen Markdown-Bericht zusammengesetzt: erst der Analyseverlauf (was wurde getan), dann die Ergebnisse und Erkenntnisse (was wurde gefunden).

6 Benutzeroberfläche (Gradio)

Das Gradio-Interface kapselt den gesamten Workflow in ca. 20 Zeilen Code. Es bietet einen Datei-Upload (CSV, Excel, TSV), eine Markdown-Ausgabe für den Bericht und eine vorbereitete Beispieldatei mit 30 Zeilen Verkaufsdaten (inklusive zwei Datensätzen mit absichtlich fehlenden Werten) als Schnellstart.

Gradio wurde gegenüber Flask oder FastAPI bevorzugt, weil es für Prototypen deutlich schneller aufzusetzen ist, direkt im Jupyter Notebook läuft und mit `share=True` einen temporären öffentlichen Link erzeugen kann.

7 LLM-Konfiguration

Als Sprachmodell wird Qwen3-Max über die DeepInfra-API eingesetzt. Die Anbindung erfolgt über die `ChatOpenAI`-Klasse von LangChain, wobei der API-Endpunkt auf DeepInfra umgeleitet wird (`openai_api_base`). Dies funktioniert, weil DeepInfra eine OpenAI-kompatible API bereitstellt.

Die Konfiguration setzt `temperature=0` für deterministische, faktenbasierte Antworten und `max_tokens=5000`. Matplotlib wird auf das Agg-Backend gesetzt, damit Plots als Dateien gespeichert statt in einem GUI-Fenster angezeigt werden.

8 Ausblick: Erweiterungsstufen

Das Notebook skizziert drei weitere Ausbaustufen:

- Stufe 2 – Reflexionsschleife: Ein Qualitäts-Agent bewertet die Ergebnisse des Analysten (Reflection Pattern). Bei einem Score unter 0,7 wird die Analyse wiederholt (max. 3x).